

A 喜迎北航 70 周年校庆！

时间限制：1000ms 内存限制：65536kb

通过率：184/192 (95.83%) 正确率：184/755 (24.37%)

本题希望同学们学会使用 C++ STL (标准模板库) 中的 `priority_queue` (优先队列)，更多请见下文的 Hint。

题目描述

Zhoues 虽然求出了钢条切割方案，使得总销售价格最大，但是真正进行钢条切割的时候，其实还有加工费用需要考虑。

最近钢条厂的生意很红火，尤其正值北航 70 周年校庆，各种英寸的钢条供不应求。现在钢条厂接到北航一份大的订单，需要 n 根钢条，订单中第 i ($1 \leq i \leq n$) 根钢条的长度是 l_i 英寸。

钢条厂有一种电锯，每开动一次可以把一根钢条锯成两段，锯出 n 根钢条就需要开动 $n - 1$ 次电锯，但是每次电锯开动都需要电费等加工费用，而且加工费用是需要锯开的钢条长度的两倍，比如要把一段长 3 英寸的钢条锯开，就要收 6 元。Zhoues 发现不同的切割次序会产生不同的费用，请帮 Zhoues 设计一个完成订单的钢条切割顺序方案，使得钢条厂付出的加工费用最少。

输入格式

第一行一个正整数 n ($1 \leq n \leq 2 \times 10^5$)，表示订单中需要钢条的个数。

接下来一行 n 个正整数 $l_1, l_2, \dots, l_n$ ($1 \leq l_i \leq 5 \times 10^4$)，表示订单中每一根钢条需要的长度。

输出格式

一行一个正整数，表示钢条厂付出的最少加工费用。

输入样例

```
3
8 5 8
```

输出样例

```
68
```

```
#include <iostream>
#include <algorithm>
#include <queue>
using namespace std;
priority_queue<long long, vector<long long>, greater<long long>> q;
long long n, i, tmp, a, b;
long long ans;
void init()
{
    ans = 0;
    while (!q.empty())
        q.pop();
}
int main()
{
    scanf("%lld", &n);
    init();
    for (i = 0; i < n; i++)
    {
        scanf("%lld", &tmp);
        q.push(tmp);
    }
    while (q.size() != 1)
    {
        a = q.top();
        q.pop();
        ans += 2*a;
        b = q.top();
```

```

        q.pop();
        ans += 2*b;
        q.push(a + b);
    }
    printf("%lld\n", ans);
    return 0;
}

```

2

B DNA 配对

时间限制：1000ms 内存限制：65536kb

通过率：190/193 (98.45%) 正确率：190/316 (60.13%)

题目描述

Zhoues 沉迷于医学实验，尤其是 DNA 配对。由于他最近一直在为北航校庆切钢条，因此 DNA 配对实验也停滞不前。他现在遇到了一个实验难题，希望大家可以帮他一下。具体来说，给定两个由 A, T, C, G 组成的字符串，请你求出它们的最长公共子序列。

输入格式

共两行，每行一个由 A, T, C, G 组成的字符串，长度均不超过 10^3 。两个字符串的长度不一定相同，且最长公共子序列的长度至少为 1。

输出格式

一行一个字符串，表示两个字符串的最长公共子序列。如果有多种可能的答案，你可以输出任意一种。

输入样例

```

AGCTTCGA
AGTTCAGC

```

输出样例

```

AGTTCA

```

```

#include<iostream>
#include<string>
#include<cstring>
#include<algorithm>
#include<set>
using namespace std;
char A[1100],B[1100];
int dp[1100][1100];
int n,m;
int targetlen;
set<string> ans;
bool flag = false;
void LCS() {
    for(int i = 1; i<=n; i++) {
        for(int j = 1; j<=m; j++) {

            if(A[i] == B[j]) {
                dp[i][j] = dp[i-1][j-1] + 1;
            } else {
                dp[i][j] = max(dp[i-1][j],dp[i][j-1]);
            }
        }
    }
    targetlen = dp[n][m];
}

```

```

void dfs(int i, int j, string s) {

    if(i<=0 || j<=0 || flag)    return ;
    if(A[i] == B[j]) {
        s.push_back(A[i]);
        if(s.length() == targetlen) {
            reverse(s.begin(),s.end());
            ans.insert(s);
            flag = true;
        } else    dfs(i-1,j-1,s);
    } else {
        if(dp[i-1][j] >= dp[i][j-1])    dfs(i-1,j,s);
        if(dp[i-1][j] <= dp[i][j-1])    dfs(i,j-1,s);
    }
}

int main() {
    int k;
    scanf("%s%s",A+1,B+1);
    n = strlen(A+1);
    m = strlen(B+1);
    LCS();
    dfs(n,m,"");
    for(string s : ans) {
        puts(s.c_str());
        break;
    }
    return 0;
}

```

3

C 奇怪的 OBST

时间限制：1000ms 内存限制：65536kb

通过率：181/182 (99.45%) 正确率：181/321 (56.39%)

题目描述

对于集合 $\{1, 2, \dots, n\}$ 的一棵二叉搜索树，定义一个节点的深度为该节点到根节点的距离，设权值为 i 的点深度为 d_i ，给定序列 $p_1, p_2, \dots, p_n$ ，定义该二叉搜索树的代价和为：

$$\sum_{i=1}^n (d_i + 1) p_i$$

请你求解集合 $\{1, 2, \dots, n\}$ 所有二叉搜索树中最小的代价和。

输入格式

第一行一个正整数 n ($1 \leq n \leq 500$)，含义同题目描述。

第二行 n 个正整数 $p_1, p_2, \dots, p_n$ ($1 \leq p_i \leq 10^4$)，含义同题目描述。

输出格式

一行一个正整数，表示最小的代价和。

输入样例

```

5
15 10 5 10 20

```

输出样例

```

130

```

```
#include<iostream>
```

```

#define maxn 1010
typedef long long LL;
using namespace std;
LL e[maxn][maxn], w[maxn][maxn], p[maxn], q[maxn], n;
void OBST()
{
    for(LL i=1;i<=n+1;i++)
        e[i][i-1] = w[i][i-1] = q[i-1];
    for(LL k=1;k<=n;k++)
    {
        for(LL i=1;i<=n-k+1;i++)
        {
            LL j = i+k-1;
            e[i][j] = 0x7fffffff;
            w[i][j] = w[i][j-1] + p[j] + q[j];
            for(LL r = i;r<=j;r++)
            {
                LL t = e[i][r-1] + e[r+1][j] + w[i][j];
                if(t < e[i][j]) e[i][j] = t;
            }
        }
    }
}

int main(){
    scanf("%lld",&n);
    for(LL i = 1;i<=n;i++) scanf("%lld",&p[i]);
    for(LL i = 0;i<=n;i++) scanf("%lld",&q[i]);
    OBST();
    printf("%lld\n",e[1][n]);
    return 0;
}

```

4

D 小水獭和切钢条

时间限制：5000ms 内存限制：65536kb

通过率：16/22 (72.73%) 正确率：16/136 (11.76%)

题目描述

互联网寒冬袭来，小水獭也打算提前进厂，体验一下流水线的感觉是什么样的。更为重要的是，北航下的钢条订单太大以至于切钢条的人手不够啦！

小水獭也给钢条厂发送了一份自己的简历，和 Zhoues 的经历类似，厂里负责招聘的工作人员正好最近也遇到了一个难题，需要一个会动态规划算法的同学帮帮忙。但与 Zhoues 不同的是，需要解决的问题发生了小小的变化：

- 给定一段长度为 n 英寸的钢条和一个价格表，该价格表表示仅有长度不超过 m 英寸的钢条，其中 i ($i = 1, 2, \dots, m$) 英寸的钢条的价格为 p_i 。求将钢条切割为 $\left\lfloor \frac{n}{m} \right\rfloor, \left\lfloor \frac{n}{m} \right\rfloor + 1, \dots, n$ 段的方案，使得总销售价格最大，注意钢条的长度必须为整英寸。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 5 \times 10^3, 1 \leq m \leq \min(n, 10)$)，含义同题目描述。

第二行 m 个正整数 $p_1, p_2, \dots, p_m$ ($1 \leq p_i \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行 $n - \left\lfloor \frac{n}{m} \right\rfloor + 1$ 个正整数，表示将钢条切割为 $\left\lfloor \frac{n}{m} \right\rfloor, \left\lfloor \frac{n}{m} \right\rfloor + 1, \dots, n$ 段的最大总销售价格。

输入样例

输入样例

```
2
4 4
1 5 8 9
10 10
1 5 8 9 10 17 17 20 24 30
```

输出样例

```
9 10 7 4
30 26 27 26 25 22 19 16 13 10
```

样例解释

```
#include <bits/stdc++.h>
#define maxn 5086

using namespace std;

typedef long long ll;

int t, n, m;
int a[maxn];
ll f[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= m; i++) scanf("%d", &a[i]);
        for(int i = 1; i <= n; i++) f[i] = -1e18;
        f[0] = 0;
        for(int i = 1; i <= n; i++){
            for(int j = n; j >= 0; j--){
                f[j] = -1e18;
                for(int l = 1; l <= m && j - l >= 0; l++) f[j] = max(f[j], f[j - l]
+ a[l]);
            }
            if(i >= (n + m - 1) / m) printf("%lld ", f[n]);
        }
        puts("");
    }
}
```

E 小水獭和求解器

时间限制：1000ms 内存限制：65536kb

通过率：11/13 (84.62%) 正确率：11/17 (64.71%)

本题提交显示 Other Error 代表 Wrong Answer。

题目描述



小水獭正在学习《高级算法分析与设计》，但遇到了一点点小问题，你能帮帮它吗？

具体来说，给定 4 个序列 $[a_1, a_2, \dots, a_m], [b_1, b_2, \dots, b_m], [c_1, c_2, \dots, c_m], [d_1, d_2, \dots, d_m]$ ，请你求出一个序列 $[x_1, x_2, \dots, x_n]$ ，满足 $x_i \in \{0, 1\}$ 对 $1, 2, \dots, n$ 成立，且在如下 m 个不等式中有不少于 $\left\lceil \frac{m}{2} \right\rceil$ 个不等式成立：

$$\begin{aligned} (x_{a_1} \oplus b_1) + (x_{c_1} \oplus d_1) &\geq 1 \\ (x_{a_2} \oplus b_2) + (x_{c_2} \oplus d_2) &\geq 1 \\ &\dots \\ (x_{a_m} \oplus b_m) + (x_{c_m} \oplus d_m) &\geq 1 \end{aligned}$$

其中 $\oplus$ 表示按位异或运算。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^4$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n, m \leq 50$)，含义同题目描述。

接下来 m 行，每行四个非负整数 a_i, b_i, c_i, d_i ($1 \leq a_i, c_i \leq n, 0 \leq b_i, d_i \leq 1$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个长度为 n 的 01 串，从左到右第 i ($1 \leq i \leq n$) 位表示 x_i 的取值。如果有多种可能的答案，你可以输出任何一种。

输入样例

```
2
4 2
1 0 2 0
3 1 4 1
2 2
1 0 1 0
1 1 1 1
```

输出样例

```
1100
11
```

```
#include <bits/stdc++.h>
#define maxn 55

using namespace std;

int t, n, m;
int a[maxn], b[maxn], c[maxn], d[maxn];
bool vis[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= m; i++) scanf("%d%d%d%d", &a[i], &b[i], &c[i], &d[i]),
vis[i] = false;
        for(int i = 1; i <= n; i++){
            int cnt[2] = {0};
            for(int j = 1; j <= m; j++) if(!vis[j]){
                if(a[j] == i) cnt[b[j]]++, vis[j] = true;
                if(c[j] == i) cnt[d[j]]++, vis[j] = true;
            }
        }
    }
}
```

```

        printf("%d", cnt[0] > cnt[1]);
    }
    puts("");
}
}

```

6

F Build Your Tree

时间限制: 1000ms 内存限制: 65536kb
 通过率: 10/12 (83.33%) 正确率: 10/38 (26.32%)

题目描述

the_ignorant 有一棵 n 个结点的树。但是这棵树太过繁密，他想要去繁为简，但同时又想保持一种整齐的美感。因此，他希望你能够将其删除若干条边，使得剩余连通块的结点数均为 k 。

若存在这样的一种方案，请你输出 **YES**，否则输出 **NO**。

输入格式

第一行一个正整数 t ($1 \leq t \leq 100$)，表示数据组数。

对于每组数据，第一行两个正整数 n, k ($1 \leq k \leq n \leq 10^3$)，含义同题目描述。

第二行 $n - 1$ 个正整数 $a_2, a_3, \dots, a_n$ ($1 \leq a_i < i$)，表示树上存在边 (i, a_i) 。

输出格式

对于每组数据，输出一行：

- 若存在满足条件的方案，输出 **YES**。
- 否则，输出 **NO**。

输入样例

```

2
2 1
1
3 2
1 2

```

输出样例

```

YES
NO

```

```

#include<bits/stdc++.h>
#define N 2000

using namespace std;
vector<int>son[N];
int fa[N];
bool vis[N];
int ans[N];
void solve(int x)
{
    if(ans[x])
    {
        return;
    }
    ans[x]=1;
    for(int i=0;i<son[x].size();i++)
    {
        solve(son[x][i]);
        ans[x]+=ans[son[x][i]];
    }
}
int main()
{

```

```

int t,n,k,x;
int tmp;
scanf("%d",&t);
while(t--)
{
    tmp=0;
    scanf("%d%d",&n,&k);
    for(int i=2;i<=n;i++)
    {
        scanf("%d",&x);
        fa[i]=x;
        son[x].push_back(i);
    }
    for(int i=n;i>=1;i--)
    {
        solve(i);
    }
    for(int i=1;i<=n;i++)
    {
        if(ans[i]%k==0)
        {
            tmp++;
        }
    }
    if(n%k==0&&tmp==n/k)
    {
        printf("YE5\n");
    }
    else
    {
        printf("NO\n");
    }
    for(int i=1;i<=n;i++)
    {
        son[i].clear();
    }
    memset(ans,0,sizeof(ans));
}
}

```


G 杀戮尖塔

时间限制: 1000ms 内存限制: 65536kb

通过率: 32/33 (96.97%) 正确率: 32/105 (30.48%)

题目描述

Bellalabella 喜欢玩一个叫做杀戮尖塔的游戏。游戏一共有 n 层，每层有 k 个关卡，编号为 $1, 2, \dots, k$ 。玩家每通过一关，则可以进入下一层 k 个关卡中任意一个，当通过 n 层时则会抵达 Boss 房挑战最终 Boss。

Bellalabella 觉得这个游戏太简单了，因此她给自己设置了两个 debuff：

- 相邻两层不能挑战编号相同的关卡，即如果进入了第 i ($1 \leq i < n$) 层的第 j ($1 \leq j \leq k$) 关，则不可进入第 $i+1$ 层的第 j 关。
- 共有 m 层被设置了陷阱，层数分别为 $a_1, a_2, \dots, a_m$ ，被设置陷阱的层不可进入第 k 关，即仅可以进入第 $1, 2, \dots, k-1$ 关。

请在 debuff 的影响下，Bellalabella 一共有多少条不同的路线能够抵达 Boss 房。两条路线不同当且仅当存在一层两条路线所挑战的关卡不同。

由于答案可能很大，你只需要输出答案对 998 244 353 取模后的结果。

输入格式

第一行三个整数 n, m, k ($1 \leq n \leq 10^6, 0 \leq m \leq n, 2 \leq k \leq 10^9$)，含义同题目描述。

第二行 m 个互不相同的正整数 $a_1, a_2, \dots, a_m$ ($1 \leq a_i \leq n$)，含义同题目描述。

输出格式

一行一个非负整数，表示不同路线数对 998 244 353 取模后的结果。

输入样例 1

```
3 1 2
2
```

输出样例 1

```
1
```

```
#include <bits/stdc++.h>
#define inf 0x3f3f3f3f
#define eps 1e-14
#define maxn 1000005
#define ls (tot << 1)
#define rs (tot << 1 | 1)
#define PII pair<int, int>
#define fi first
#define se second
typedef long long ll;
typedef long double ld;
typedef unsigned long long ull;
using namespace std;
const double pi = acosl(-1.0L);
const int mod = 998244353;
inline int read(){
    int x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

int n, m, k;
bool vis[maxn];
int dp[maxn][2];
set<int> s;

void solve(){
    n = read(), m = read(), k = read();
    for(int i = 1; i <= m; i++) vis[read()] = 1;
    dp[1][0] = k - 1, dp[1][1] = !vis[1];
    for(int i = 2; i <= n; i++){
```

```

dp[i][0] = (111 * dp[i - 1][0] * (k - 2) + 111 * dp[i - 1][1] * (k - 1))
% mod;

dp[i][1] = dp[i - 1][0];
if(vis[i]) dp[i][1] = 0;
}

printf("%d", (dp[n][0] + dp[n][1]) % mod);
}

int main(){
    int t = 1, cnt = 1;
    while(t--){
        solve();
    }

    return 0;
}

```

H XIAO7 和数集 (困难版)

由于所有的喵币都分给「雪风」和「星瑶」了，猫娘「凉宫秋小七」对此很不高兴又哭又闹，XIAO7 只好告诉她，若你能答对以下问题，我就把上一题扔掉的喵币捡回来偷偷给你。

- $S(11523) = \{1, 2, 3, 5\}$
- $S(67654) = \{4, 5, 6, 7\}$

输入格式

对于每组数据，一行一个正整数 n ($1 \leq n < 10^{10^3}$)，含义同题目描述。

输出格式

输入样例

输出样例

```
#include <bits/stdc++.h>
#define maxn 1086

using namespace std;

const int p = 998244353;

int t, n;
char s[maxn];
int f[maxn][1 << 10][2][2];

void add(int &x, int y){
    x += y;
    if(x >= p) x -= p;
}
```

```

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%s", s + 1);
        n = strlen(s + 1);
        for(int i = 1;i <= n;i++) for(int j = 0;j < 1024;j++) for(int k = 0;k <=
1;k++) for(int l = 0;l <= 1;l++) f[i][j][k][l] = 0;
        f[0][0][1][0] = 1;
        for(int i = 1;i <= n;i++) for(int j = 0;j < 1024;j++) for(int k = 0;k <=
9;k++) for(int l = 0;l <= 1;l++){
            int jj = j | (1 << k), ll = l | (k > 0);
            if(!ll) jj = 0;
            add(f[i][jj][0][ll], f[i - 1][j][0][ll]);
            if(k <= s[i] - '0') add(f[i][jj][k == s[i] - '0'][ll], f[i - 1][j][1]
[ll]);
        }
        int ans = 0;
        for(int i = 1;i < 1024;i++) add(ans, (f[n][i][0][1] + f[n][i][1][1]) *
(1ll * f[n][i][0][1] + f[n][i][1][1] + p - 1) % p * (p + 1 >> 1) % p);
        printf("%d\n", ans);
    }
}

```

9

I XIAO7 和最长上升子序列

时间限制: 1000ms 内存限制: 204800kb

通过率: 2/4 (50.00%) 正确率: 2/11 (18.18%)

本题提交显示 Other Error 代表 Wrong Answer。

题目描述

XIAO7 在算法课上学会了如何计算最长上升子序列，但她觉得这对于猫猫来说实在是太简单了，于是她决定把问题升级一下来考核她的猫猫们。

具体来说，给定长度为 n 的序列 $f_1, f_2, \dots, f_n$ ，请你求解如下两个问题：

1. 构造一个 1 到 n 的排列 $a_1, a_2, \dots, a_n$ 满足以 a_i 结尾的最长上升子序列的长度为 f_i 。
2. 对于 $i = 1, 2, \dots, n$ ，求解所有满足第一个问题的排列中， a_i 的最大值。

对于一个序列 $a_1, a_2, \dots, a_n$ 来说，若序列 $x_1, x_2, \dots, x_m$ 满足 $1 \leq x_1 < x_2 < \dots < x_m \leq n$ 且 $a_{x_1} < a_{x_2} < \dots < a_{x_m}$ 则称 $a_{x_1}, a_{x_2}, \dots, a_{x_m}$ 为一个上升子序列。最长上升子序列是指所有上升子序列中长度最大的上升子序列。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^5$)，含义同题目描述。

第二行 n 个正整数 $f_1, f_2, \dots, f_n$ ($1 \leq f_i \leq i$)，含义同题目描述。

保证至少存在一个排列满足第一个问题的条件。

输出格式

第一行一个 1 到 n 的排列，表示第一个问题的答案。如果有多种可能的答案，你可以输出任意一种。

第二行 n 个正整数，第 i ($1 \leq i \leq n$) 个整数表示所有满足第一个问题的排列中， a_i 的最大值。

输入样例

```

3
1 1 2

```

输出样例

```

2 1 3
3 1 3

```

```

#include<iostream>
#include<cstdio>
#include<cmath>

```

```

#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
using namespace std;
typedef long long ll;
const int N = 1000005;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
int f[N], fa[N], layer[N], ans[N], sum[N], sz[N], a[N];
vector<int> son[N];
vector<int> layer_point[N];
void dfs(int x){
    sz[x] = 1;
    int m = son[x].size();
    for(int i = 0; i < m; i++){
        dfs(son[x][i]);
        sz[x] += sz[son[x][i]];
    }
    ans[x] = sum[f[x]] + sz[x] - 1;
    sum[f[x]] += sz[x];
}
int main(){
    int n = getint();

    for(int i = 1; i <= n; i++){
        f[i] = getint();

        for(int i = 1; i <= n; i++){
            fa[i] = layer[f[i] - 1];
            layer[f[i]] = i;
            layer_point[f[i]].push_back(i);
            son[fa[i]].push_back(i);
        }
        dfs(0);

        for(int i = 1, j = 0; i <= n; i++){
            int k = (j += layer_point[i].size());
            for(auto x : layer_point[i])
                a[x] = k--;
        }
        for(int i = 1; i <= n; i++){
            printf("%d%c", a[i], i == n ? 10 : 32);

            for(int i = 1; i <= n; i++){
                printf("%d%c", n - ans[i], i == n ? 10 : 32);
            }
        }
        exit(0);
    }
}

```

J 小水獭和体育馆（困难版）

时间限制：15000ms 内存限制：204800kb

通过率：1/3 (33.33%) 正确率：1/15 (6.67%)

题目描述



《与国同行》建校 70 周年庆祝晚会将于 10 月 24 日（预演、备播录制）、10 月 25 日（正式演出）进行两场演出，演出地点沙河体育馆，每晚 18:30 开始检票。

小水獭正在学习动态规划，但想去看看《与国同行》放松一下。

体育馆可以被抽象为一个 $n \times m$ 的矩阵 $\{a_{i,j}\}_{n \times m}$ ，如果体育馆第 i 行第 j 列的座位有人，则 $a_{i,j} = 1$ ，否则 $a_{i,j} = 0$ 。

众所周知，为防控疫情，相邻座位不能同时有人。形式化地说， $(a_{i,j} = 1 \wedge a_{i',j'} = 1) \Rightarrow (|i - i'| + |j - j'| \neq 1)$ 。

此外，有 k 个额外限制某个座位必须有人或没人。形式化地说，第 i 个限制开闭为 $a_{x_i,y_i} = z_i$ 。

小水獭想知道，有多少种不同的矩阵所对应的体育馆满足疫情防控要求和 k 个额外限制，由于答案可能很大，你只需要输出答案对 2^{32} 取模后的结果。

输入格式

第一行一个正整数 t ($1 \leq t \leq 3$)，表示数据组数。

对于每组数据，第一行三个整数 n, m, k ($1 \leq nm \leq 625, 1 \leq k \leq 10$)，含义同题目描述。

接下来 k 行，第 i 行三个整数 x_i, y_i, z_i ($1 \leq x_i \leq n, 1 \leq y_i \leq m, 0 \leq z_i \leq 1$)，表示第 i 个限制的三个参数。

输出格式

对于每组数据，输出一行一个非负整数，表示答案对 2^{32} 取模后的结果。

输出格式

对于每组数据，输出一行一个非负整数，表示答案对 2^{32} 取模后的结果。

输入样例

```
2
2 2 1
1 1 0
6 6 3
1 1 1
2 2 0
3 3 1
```

输出样例

```
5
231661
```

```
#include <bits/stdc++.h>
#define maxn 200086

using namespace std;

int t, n, m, k;
map<int, int> mp;
int cnt;

void dfs(int i, int j){
    if(i & (i << 1)) return;
    if(j == m){
        mp[i] = ++cnt;
        return;
    }
    dfs(i, j + 1), dfs(i | (1 << j), j + 1);
```

```

}

int a[maxn][30][2];
unsigned f[maxn], g[maxn];
int b[1005][1005];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d%d", &n, &m, &k);
        bool tag = false, flag = false;
        if(n < m) swap(n, m), tag = true;
        for(int i = 0; i < n; i++) for(int j = 0; j < m; j++) b[i][j] = -1;
        while(k--){
            int x, y, z;
            scanf("%d%d%d", &x, &y, &z);
            if(tag) swap(x, y);
            x--, y--;
            if(b[x][y] == -1) b[x][y] = z;
            else if(b[x][y] != z) flag = true;
        }
        if(flag){
            puts("0");
            continue;
        }
        mp.clear();
        cnt = 0, dfs(0, 0);
        for(auto it : mp){
            int s = it.first, x = it.second;
            for(int i = 0; i < m; i++){
                a[x][i][0] = mp[s & ~(1 << i)];
                if((i == 0 || !(s & (1 << (i - 1)))) && !(s & (1 << i))){
                    a[x][i][1] = mp[(s | (1 << i)) & ~(1 << (i + 1))];
                }else{
                    a[x][i][1] = 0;
                }
            }
        }
        for(int i = 1; i <= cnt; i++) f[i] = 0;
        f[mp[0]] = 1;
        for(int i = 0; i < n; i++) for(int j = 0; j < m; j++){
            for(int k = 1; k <= cnt; k++){
                if(b[i][j] != 1) g[a[k][j][0]] += f[k];
                if(b[i][j] != 0 && a[k][j][1]) g[a[k][j][1]] += f[k];
            }
            for(int k = 1; k <= cnt; k++) f[k] = g[k], g[k] = 0;
        }
        unsigned ans = 0;
        for(int i = 1; i <= cnt; i++) ans += f[i];
        printf("%u\n", ans);
    }
}

```

A 购物

时间限制: 1000ms 内存限制: 65536kb

通过率: 190/195 (97.44%) 正确率: 190/1064 (17.86%)

题目描述

虽然 Zhoues 习惯在学校和钢厂之间来回奔波，但是每到周末还是需要放松一下的，于是 Zhoues 和小水獭打算周末一起去购物放松心情。

Zhoues 手中有一份地图，地图上每一个购物中心都有一个唯一的编号，而且地图上也会标注出每两个购物中心之间的距离。但是因为最近车流量大，地图上的双行道都变成单行道以缓解交通压力。小水獭想知道给定任意两个购物中心，二者之间的最短路径长度是多少。

形式化来说，给定 n 个点 m 条的有向图，你需要处理 q 次询问，第 i 次询问的内容为点 x_i 到点 y_i 的最短路径长度（若不可达则输出 `No Solution`）。

输入格式

第一行三个正整数 n, m, q ($3 \leq n \leq 500, 1 \leq m, q \leq 2.5 \times 10^5$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^7$)，表示第 i 条边 $u_i \rightarrow v_i$ 的权值为 w_i 。

接下来 q 行，每行两个正整数 x_i, y_i ($1 \leq x_i, y_i \leq n$)，含义同题目描述。

图中有可能存在重边和自环。

输出格式

输出 q 行，第 i ($1 \leq i \leq q$) 行表示第 i 次询问的答案：

- 如果点 x_i 可以到点 y_i ，则输出最短路径长度。
- 否则，输出 `No Solution`。

输入样例

```
3 3 2
1 2 3
2 3 1
3 1 2
1 2
1 3
```

输出样例

```
3
4
```

```
#include<bits/stdc++.h>
const int MN=500+5;
typedef long long ll;
using namespace std;
ll mat[MN][MN];
ll inf = 0x3f3f3f3f3f3f3f3f;
int m,n,p;
int main()
{
    memset(mat,0x3f,sizeof mat);
    scanf("%d%d%d",&n,&m,&p);
    ll w;
    for(int i=1,u,v;i<=m;i++){
        scanf("%d%d%lld",&u,&v,&w);
        mat[u][v]=min(mat[u][v],w); //Öö±ß
    }
    for(int i=1;i<=n;i++){
        mat[i][i]=0; // ×Ö»·
    }
    for(int k=1;k<=n;k++)
        for(int i=1;i<=n;i++)
            for(int j=1;j<=n;j++)
                mat[i][j]=min(mat[i][j],mat[i][k]+mat[k][j]);

    for(int i=0,u,v;i<p;i++){
        scanf("%d%d",&u,&v);
```

```

        if(mat[u][v]<inf) printf("%lld\n",mat[u][v]);
        else puts("No Solution");
    }
    return 0;
}

```

2

B 铺光缆

时间限制：1000ms 内存限制：65536kb

通过率：179/195 (91.79%) 正确率：179/1077 (16.62%)

题目描述

罗川老师在担任 21 级的「算法分析与设计」老师的同时，也担任 20 级的「计算机网络」老师。罗老师有一次在讲解计算机网络中物理层的时候，提到了北航光缆铺设问题，问题大致如下：

- 北航主要办公科研楼有新主楼、逸夫楼、如心楼、办公楼、图书馆、主楼、一号楼等等。北航网络中心计划要给相关建筑物间铺设光缆进行网络连通。给定建筑物之间的距离，计算铺设光缆用料最少是多少。

Zhoues 其实在瞬间就计算完毕了，但是他不确定是不是正确的。于是他希望你帮忙计算一下铺设光缆用料最少是多少。

形式化地说，给定一个 n 个点 m 条边的带权无向连通图，请你求解最小生成树的边权之和。

输入格式

第一行三个正整数 n, m ($1 \leq n \leq 2 \times 10^5$, $n - 1 \leq m \leq 5 \times 10^5$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n$, $1 \leq w_i \leq 10^9$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

图中有可能存在重边和自环，保证图是连通的。

输出格式

输出一行一个正整数，表示最小生成树的边权之和。

输入样例

```

3 3
1 2 1
1 3 2
2 3 1

```

输出样例

```

2

```

```

#include <bits/stdc++.h>
#define mp make_pair
#define ms(a,b) memset(a,b,sizeof(a))
#define maxn 1000007
typedef long long LL;
using namespace std;

struct edge
{
    LL u,v;
    LL w;
};

struct edge edges[60000100];
LL i;
LL Father[1010000];
LL cnt;
long long res;
void initFather(LL vertexNum)
{
    LL i;
    for(i=1;i<=vertexNum;++i)
    {
        Father[i]=i;
    }
}

```



```

    }
}
LL getFather(LL x)
{
    return Father[x]==x?(Father[x]=getFather(Father[x])):
}
void kruskal(LL vertexNum,LL edgeNum)
{
    LL p,q;
    cnt=0,res=0;

    for(i=0;i<edgeNum;++i)
    {
        p=getFather(edges[i].u);
        q=getFather(edges[i].v);
        if(p!=q)
        {
            Father[p]=q;
            res+=edges[i].w;
            cnt++;
        }
        if(cnt==vertexNum-1)
        {
            break;
        }
    }
}

}
int cmp(const void*p1,const void*p2)
{
    struct edge *a=(struct edge*)p1;
    struct edge *b=(struct edge*)p2;
    return a->w-b->w;
}
int main()
{
    LL n,m;
    scanf("%lld%lld",&n,&m);
    initFather(n);
    LL i;
    for(i=0;i<m;++i)
        scanf("%lld %lld %lld",&edges[i].u,&edges[i].v,&edges[i].w);
    qsort(edges,m,sizeof(struct edge),cmp);
    kruskal(n,m);
    printf("%lld\n",res);
}

```

C 上班 II

时间限制: 2000ms 内存限制: 204800kb

通过率: 173/190 (91.05%) 正确率: 173/1226 (14.11%)

题目描述

观赛完运动会之后, Zhoues 需要立马回去钢厂上班, 由于从家到钢厂的路径有很多条, 他希望选择一条最短的!

但是现在 Zhoues 很急, 所以希望可以计算的更快一点!

形式化来说, 给一个 n 个点 m 条边的无向图, 求点 s 到点 t 的最短路长度, 题目保证至少存在一条点 s 到点 t 的路径。

输入格式

第一行四个正整数 n, m, s, t ($1 \leq n \leq 10^6, 1 \leq m \leq 1.6 \times 10^6, 1 \leq s, t \leq n$), 含义同题目描述。

接下来 m 行, 每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^7$), 表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

图中有可能存在重边和自环, 保证至少存在一条点 s 到点 t 的路径。

输出格式

输出一行一个非负整数, 表示点 s 到点 t 的最短路长度。

输入样例

```
7 11 5 4
2 4 2
1 4 3
7 2 2
3 4 3
5 7 5
7 3 3
6 1 1
6 3 4
2 4 3
5 6 3
7 2 1
```

输出样例

```
7
```

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll read() {
    ll x=0,w=1; char c=getchar();
    while (c<'0' || c>'9') { if (c=='-') w=-1; c=getchar(); }
    while (c>='0' && c<='9') x=x*10+c-'0', c=getchar();
    return x*w;
}

const ll N=10000000+10, M=15000000+10;

ll n,m,s,t;
struct edge { ll v,w,nxt; } e[M];
ll head[N];
void addEdge(ll u,ll v,ll w) {
    static ll cnt=0;
    e[++cnt]=(edge){v,w,head[u]}, head[u]=cnt;
}

priority_queue<pair<ll,ll>,vector<pair<ll,ll>>,greater<pair<ll,ll>>> Q;
ll dis[N];
void dijkstra(ll s) {
    memset(dis,0x3f,sizeof(dis));
    dis[s]=0,Q.push(make_pair(0,s));
    while (!Q.empty()) {
        ll u=Q.top().second,d=Q.top().first; Q.pop();
```

```

        if (d!=dis[u]) continue;
        for (ll i=head[u];i;i=e[i].nxt) {
            ll v=e[i].v,w=e[i].w;
            if (dis[u]+w<dis[v]) dis[v]=dis[u]+w,Q.push(make_pair(dis[v],v));
        }
    }
}

int main() {
    n=read(),m=read(),s=read(),t=read();
    for (ll i=1;i<=m;++i) {
        ll u=read(),v=read(),w=read();
        addEdge(u,v,w);
        addEdge(v,u,w);
    }
    dijkstra(s);
    printf("%lld",dis[t]);
    return 0;
}

```

4

D 小水獭和负环

时间限制: 1000ms 内存限制: 65536kb

通过率: 172/179 (96.09%) 正确率: 172/781 (22.02%)

题目描述



观赛完运动会之后，Zhoues 需要立马回去钢厂上班，由于北航到钢厂的路径有很多条，他希望选择一条最短的！

但是，图中可能存在负环，Zhoues 不想无穷循环下去，因此他找到小水獭帮忙，希望小水獭可以帮它判断是否有负环。

形式化来说，给你一个 n 个点 m 条边的带权有向图，如果图中存在负环，你需要告知 Zhoues，否则你需要求解从点 1 到所有点的最短路（如果不可达，认为最短路长度是 10^9 ）。

负环是指一条边权之和为负数的回路。

输入格式

第一行一个正整数 t ($1 \leq t \leq 20$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 2 \times 10^3$, $1 \leq m \leq 6 \times 10^3$)，含义同题目描述。

接下来 m 行，每行三个整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n$, $-10^4 \leq w_i \leq 10^4$)，表示第 i 条边 $u_i \rightarrow v_i$ 的权值为 w_i 。

图中有可能存在重边和自环。

输出格式

对于每组数据，输出一行：

- 如果图中存在负环，输出 `boo how`。
- 否则输出 n 个整数，表示从点 1 到点 $1, 2, \dots, n$ 的最短路长度（如果不可达，认为最短路长度是 10^9 ）。

输入样例

输入样例

```
2
4 5
1 2 -1
2 3 0
3 4 0
4 1 1
4 1 0
4 1
1 2 1
```

输出样例

```
boo how
0 1 1000000000 1000000000
```

```
#include <bits/stdc++.h>
#define maxn 2086

using namespace std;

int t, n, m;
vector<pair<int, int> > v[maxn];
int d[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= n; i++) v[i].clear(), d[i] = 1e9;
        while(m--){
            int x, y, z;
            scanf("%d%d%d", &x, &y, &z);
            v[x].push_back({y, z});
        }
        d[1] = 0;
        bool tag = false;
        for(int j = 1; j <= n; j++){
            for(int i = 1; i <= n; i++){
                for(auto it : v[i]){
                    int to = it.first, co = it.second;
                    if(d[to] > d[i] + co && j == n) tag = true;
                    d[to] = min(d[to], d[i] + co);
                }
            }
        }
        if(tag) puts("boo how");
        else{
            for(int i = 1; i <= n; i++) printf("%d ", d[i]); puts("");
        }
    }
}
```

E 小水獭和哈密顿路径

时间限制: 1000ms 内存限制: 65536kb

通过率: 156/167 (93.41%) 正确率: 156/629 (24.80%)

题目描述



众所周知，判断一个图是否有哈密顿路径是 **NP** 的。
但小水獭发现，如果一个图是 DAG，那么这个问题是 **P** 的，你能帮它实现一下嘛？
形式化来说，给一个 n 个点 m 条边的有向无环图，请你判断图中是否存在一条哈密顿路径。
哈密顿路径是指一条经过每个点恰好一次的路径。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。
对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$)，含义如题目所示。
接下来 m 行，每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示第 i 条边为 $u_i \rightarrow v_i$ 。
保证图是有向无环图，且没有重边。

输出格式

对于每组数据，输出一行：

- 如果图中存在哈密顿路径，输出 `yy`。
- 否则输出 `nn`。

输入样例

```
2
4 4
1 2
1 3
2 3
3 4
4 3
1 2
1 3
1 4
```

输出样例

```
yy
nn
```

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n, m;
vector<int> v[maxn];
int d[maxn];

int dfs(int i){
    if(d[i] != -1) return d[i];
    d[i] = 0;
    for(auto to : v[i]){
        d[i] = max(d[i], dfs(to));
    }
    return ++d[i];
}

int main(){
    scanf("%d", &t);
    while(t--){
```

```

scanf("%d%d", &n, &m);
for(int i = 1; i <= n; i++) v[i].clear(), d[i] = -1;
while(m--){
    int x, y;
    scanf("%d%d", &x, &y);
    v[x].push_back(y);
}
int ans = 0;
for(int i = 1; i <= n; i++) ans = max(ans, dfs(i));
puts(ans == n ? "yy" : "nn");
}
}

```

6

F 这下不得不进厂了

时间限制: 1000ms 内存限制: 65536kb

通过率: 157/167 (94.01%) 正确率: 157/752 (20.88%)

题目描述

这几周 Zhoues 一直在钢厂上班，the_ignorant 却啥也没干，他感到了强烈的危机感，他急了，这下不得不进厂了！

the_ignorant 进了自来水厂，可他刚入职就碰到了一个难题。在水管分布图中，有 n 个阀门和 m 个单向管道，每个单向管道连接两个阀门，并且有各自的容量限制（即经过的水流不能超过该管道的最大容量）。the_ignorant 想知道，在阀门 s 放水时，阀门 t 最大能接收到多大的水流。

形式化地说，给定 n 个点 m 条边的有向图，每条边有最大容量，求从点 s 到点 t 的最大流。

输入格式

第一行一个正整数 T ($1 \leq T \leq 10$)，表示数据组数。

对于每组数据，第一行四个正整数 n, m, s, t ($1 \leq n \leq 100, 1 \leq m \leq 5 \times 10^3, 1 \leq s, t \leq n$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 0 \leq w_i < 2^{31}$)，表示第 i 条有向边 $u_i \rightarrow v_i$ 的最大容量为 w_i 。

图中有可能存在重边和自环。

输出格式

对于每组数据，输出一行一个非负整数，表示点 s 到点 t 的最大流。

输出格式

对于每组数据，输出一行一个非负整数，表示点 s 到点 t 的最大流。

输入样例

```

1
7 14 1 7
1 2 5
1 3 6
1 4 5
2 3 2
2 5 3
3 2 2
3 4 3
3 5 3
3 6 7
4 6 5
5 6 1
6 5 1
5 7 8
6 7 7

```

输出样例

```

14

```

```

#include <bits/stdc++.h>
#define mp make_pair
#define ms(a,b) memset(a,b,sizeof(a))
#define maxn 1000007
typedef long long LL;
using namespace std;
inline LL read() {
    LL x=0,w=1;
    char ch=0;

```

```

while(ch<'0' || ch>'9') {
    if(ch=='-') w=-1;
    ch=getchar();
}
while(ch>='0' && ch<='9') x=(x<<3)+(x<<1)+ch-'0',ch=getchar();
return x*w;
}

inline void write(LL x) {
    if(x < 0)putchar('-'),x=-x;
    if (x > 9)write(x / 10);
    putchar(x % 10 + 48);
}

const LL inf=0x3f3f3f3f;
const LL N=2000005;
const LL M=2000005;
LL cnt,tot;
LL head[N],d[N],ans[N];
bool vis[N];
struct Edge {
    LL v,next;
    LL cap,flow;
} E[M<<1]; //双边

void init() { //初始化
    memset(head,-1,sizeof(head));
    cnt=0;
}

void add(LL u,LL v,LL c) {
    E[cnt].v=v;
    E[cnt].cap=c;
    E[cnt].flow=0;
    E[cnt].next=head[u];
    head[u]=cnt++;
}

void adde(LL u,LL v,LL c) {
    add(u,v,c);
    add(v,u,0);
}

bool bfs(LL s,LL t) { //分层
    memset(d,0,sizeof(d));
    queue<LL>q;
    d[s]=1;
    q.push(s);
    while(!q.empty()) {
        LL u=q.front();
        q.pop();
        for(LL i=head[u]; ~i; i=E[i].next) {
            LL v=E[i].v;
            if(!d[v]&&E[i].cap>E[i].flow) {
                d[v]=d[u]+1;
                q.push(v);
                if(v==t) return 1;
            }
        }
    }
}

```

```

    }
}
return 0;
}

LL dfs(LL u, LL flow, LL t) { //在分层的基础上dfs
    if(u==t) return flow;
    LL rest=flow;
    for(LL i=head[u]; ~i&&rest; i=E[i].next) {
        LL v=E[i].v;
        if(d[v]==d[u]+1&&E[i].cap>E[i].flow) {
            LL k=dfs(v, min(rest, E[i].cap-E[i].flow), t);
            if(!k) d[v]=0;
            E[i].flow+=k;
            E[i^1].flow-=k;
            rest-=k;
        }
    }
    return flow-rest;
}

LL Dinic(LL s, LL t) {
    LL maxflow=0;
    while(bfs(s, t)) {
        maxflow+=dfs(s, inf, t);
    }
    return maxflow;
}

LL n, m, s, t, T;
int main() {
    T = read();
    while(T--) {
        n = read(), m = read(), s = read(), t = read();
        init();
        for(LL i = 0; i < m; i++) {
            LL x = read();
            LL y = read();
            LL w = read();
            add(x, y, w);
            add(y, x, 0);
        }
        write(Dinic(s, t));
        putchar('\n');
    }

    return 0;
}

```


G 旅行计划

时间限制：1500ms 内存限制：65536kb

通过率：24/36 (66.67%) 正确率：24/195 (12.31%)

题目描述

教练们很累，写不动题目背景力！

形式化地说，给定一张 n 个点 m 条边的带权无向连通图。你需要处理 q 次询问，第 i 次询问的内容为从点 s_i 到点 t_i 经过不超过 k_i 条边的最短路长度是多少。

输入格式

第一行三个整数 n, m, q ($1 \leq n \leq 100, n - 1 \leq m \leq \frac{n(n-1)}{2}, 1 \leq q \leq 10^6$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i < v_i \leq n, 1 \leq w_i \leq 10^9$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

接下来 q 行，每行三个正整数 s_i, t_i, k_i ($1 \leq s_i, t_i \leq n, 1 \leq k_i \leq m$)，含义同题目描述。

保证图连通且没有重边和自环。

输出格式

输出 q 行，第 i ($1 \leq i \leq q$) 行一个整数：

- 如果不存在一条从 s_i 到 t_i 且经过边数不超过 k 的路径，输出 -1 。
- 否则输出从点 s_i 到点 t_i 经过不超过 k_i 条边的最短路长度。

输出格式

输出 q 行，第 i ($1 \leq i \leq q$) 行一个整数：

- 如果不存在一条从 s_i 到 t_i 且经过边数不超过 k 的路径，输出 -1 。
- 否则输出从点 s_i 到点 t_i 经过不超过 k_i 条边的最短路长度。

输入样例

```
4 4 4
1 2 10
2 3 20
2 4 1000
1 4 30
1 3 1
1 3 2
2 4 1
2 4 4
```

输出样例

```
-1
30
1000
40
```

```
#include <stdio>
#include <string.h>
const int N = 105;
struct edge {
    int u, v;
    long long w;
};
edge e[N * N];
int cnt = 0;
long long dis[N][N][N];
int n, m, q, u, v, w, s, t, k;
void BF(int s) {
    for (int i = 0; i <= n - 1; i++) {
        dis[i][s][s] = 0ll;
    }
    for (int i = 1; i <= n - 1; i++) {
        for (int j = 1; j <= n; j++) {
            dis[i][s][j] = dis[i - 1][s][j];
        }
        for (int j = 1; j <= cnt; j++) {
            if (dis[i][s][e[j].v] > dis[i - 1][s][e[j].u] + e[j].w) {
```

```

        dis[i][s][e[j].v] = dis[i - 1][s][e[j].u] + e[j].w;
    }
    if (dis[i][s][e[j].u] > dis[i - 1][s][e[j].v] + e[j].w) {
        dis[i][s][e[j].u] = dis[i - 1][s][e[j].v] + e[j].w;
    }
}
}

int main() {
    scanf("%d%d%d", &n, &m, &q);
    for (int i = 1; i <= m; i++) {
        scanf("%d%d%d", &u, &v, &w);
        e[++cnt].u = u;
        e[cnt].v = v;
        e[cnt].w = (long long)w;
    }
    for (int i = 0; i <= n - 1; i++) {
        for (int j = 1; j <= n; j++) {
            for (int k = 1; k <= n; k++) {
                dis[i][j][k] = 111 << 60;
            }
        }
    }
    for (int i = 1; i <= n; i++) {
        BF(i);
    }
    for (int i = 1; i <= q; i++) {
        scanf("%d%d%d", &s, &t, &k);
        if (k >= n) {
            if (dis[n - 1][s][t] == 111 << 60) {
                printf("-1\n");
            } else {
                printf("%11d\n", dis[n - 1][s][t]);
            }
        } else {
            if (dis[k][s][t] == 111 << 60) {
                printf("-1\n");
            } else {
                printf("%11d\n", dis[k][s][t]);
            }
        }
    }
    return 0;
}

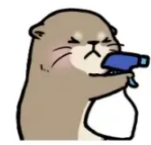
```

H 小水獭和补图

时间限制：1000ms 内存限制：65536kb

通过率：20/27 (74.07%) 正确率：20/133 (15.04%)

题目描述



小水獭正在研究补图的连通性问题。

具体来说，对于无重边自环的无向图 G ，设其节点为 $1, 2, \dots, n$ ，边集为 E ；定义它的补图为 G' ，其中 G' 的点集和 G 相同， G' 的边集为 $\{(u, v) \mid 1 \leq u, v \leq n, u \neq v, (u, v) \notin E\}$ 。

给定 n 个点 m 条边且无重边自环的无向图，小水獭想知道其补图中每个点所在连通块的大小。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$)，含义如题目所示。

接下来 m 行，每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示第 i 条边为 (u_i, v_i) 。

保证图中没有重边和自环。

输出格式

对于每组数据，输出一行 n 个正整数，表示补图中点 $1, 2, \dots, n$ 所在连通块的大小。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$)，含义如题目所示。

接下来 m 行，每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示第 i 条边为 (u_i, v_i) 。

保证图中没有重边和自环。

输出格式

对于每组数据，输出一行 n 个正整数，表示补图中点 $1, 2, \dots, n$ 所在连通块的大小。

输入样例

```
2
3 2
1 2
1 3
4 6
1 2
1 3
1 4
2 3
2 4
3 4
```

输出样例

```
1 2 2
1 1 1 1
```

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n, m;
vector<int> v[maxn];
int fa[maxn], siz[maxn];

int find(int x){
    return x == fa[x] ? x : fa[x] = find(fa[x]);
}

void merge(int x, int y){
    x = find(x), y = find(y);
    if(x == y) return;
```

```

        if(siz[x] > siz[y]) swap(x, y);
        fa[x] = y, siz[y] += siz[x];
    }

    bool vis[maxn];

    void cal(int i, int j){
        for(auto to : v[i]) vis[to] = j;
    }

    int main(){
        scanf("%d", &t);
        while(t--){
            scanf("%d%d", &n, &m);
            for(int i = 1; i <= n; i++) v[i].clear(), fa[i] = i, siz[i] = 1;
            while(m--){
                int x, y;
                scanf("%d%d", &x, &y);
                v[x].push_back(y), v[y].push_back(x);
            }
            int x = 1;
            for(int i = 2; i <= n; i++) if(v[i].size() < v[x].size()) x = i;
            cal(x, 1);
            for(int i = 1; i <= n; i++) if(!vis[i]) merge(x, i);
            cal(x, 0);
            for(auto to : v[x]){
                cal(to, 1);
                for(int i = 1; i <= n; i++) if(!vis[i]) merge(to, i);
                cal(to, 0);
            }
            for(int i = 1; i <= n; i++) printf("%d ", siz[find(i)]); puts("");
        }
    }
}

```

9

I XIAO7 不想努力了

时间限制: 1000ms 内存限制: 131072kb

通过率: 8/12 (66.67%) 正确率: 8/54 (14.81%)

题目描述

XIAO7: 「我不想努力了, 我今天创造的价值已经够多了。」

天上传来一个声音: 「如果你能答对以下问题, 你就可以不努力了。」

具体来说, 给定一个 n 个点 m 条边的有向图, 请你给所有边分组, 使得每一组内边的编号是连续的, 并且不存在一条从点 1 到点 n 的路径满足路径上所有边属于同一组。你需要求解最少需要分多少组。

请你帮帮可爱的 XIAO7!

输入格式

第一行两个正整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 10^6$), 含义同题目描述。

接下来 m 行, 每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$), 表示第 i 条边为 $u_i \rightarrow v_i$ 。

图中可能有重边, 但没有自环, 且不存在边 $1 \rightarrow n$ 。

输出格式

一行一个正整数, 表示最少的分组数。

输出格式

一行一个正整数，表示最少的分组数。

输入样例

```
3 5
2 3
3 2
2 1
1 2
3 2
```

输出样例

```
2
```

样例解释

一种分组数最少的方案为 $[1, 2, 3], [4, 5]$ 。

仅考虑第一组的边得到的图如下：

仅考虑第二组的边得到的图如下：

均不存在从点 1 到点 n 的路径。

1 2 ... n

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<ctime>
#include<assert.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 100005;
const int M = 1000005;
const int inf = 0x3f3f3f3f;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0' && c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
int cnt, fir[N], tar[M], nxt[M];
int vis[N], U[M], V[M];
void link(int a, int b) {
    tar[++ cnt] = b, nxt[cnt] = fir[a], fir[a] = cnt;
}
void dfs(int x) {
    for(int i = fir[x]; i; i = nxt[i]) {
        if(vis[tar[i]] == vis[x])
            continue ;
        vis[tar[i]] = vis[x], dfs(tar[i]);
    }
}
int main(){
    int n = getint(), m = getint();
```

```

int group = 1, last = 0;
vis[1] = group;
for(int i = 1; i <= m; i++) {
    int x = getint(), y = getint();
    link(x, y), U[i] = x, V[i] = y;
    if(vis[x] == group)
        dfs(x);
    if(vis[n] == group) {
        vis[1] = ++ group;
        for(int j = i; j >= last; j--)
            fir[U[j]] = 0;
        last = i, link(x, y);
        if(x == 1) vis[y] = group;
    }
}
cout << group << endl;
}

```

10

A B C D E F G H I J

J 小水獭和最小环

时间限制: 8000ms 内存限制: 65536kb

通过率: 2/9 (22.22%) 正确率: 2/76 (2.63%)

题目描述



小水獭正在研究无向图的最小环问题。

具体来说，给定 n 个点 m 条边的带权无向图，请你求出对于点 $i = 1, 2, \dots, n$ 来说，包含点 i 的最小环大小（如果不存在包含点 i 的环，认为最小环大小为 10^9 ）。

环是指点数至少为 3 且不重复经过相同点的回路，最小环是指边权之和最小的环。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 500, 1 \leq m \leq \frac{n(n-1)}{2}$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^6$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

保证给出的图没有重边和自环。

输出格式

对于每组数据，输出一行 n 个正整数，表示对于点 $i = 1, 2, \dots, n$ 来说，包含点 i 的最小环大小（如果不存在包含点 i 的环，认为最小环大小为 10^9 ）。

输入样例

```

2
4 4
1 2 5
1 3 1
1 4 1
2 4 1
3 3
1 2 1
2 3 2
3 1 3

```

输出样例

```

7 7 1000000000 7
6 6 6

```

```

#include <bits/stdc++.h>
#define maxn 505

using namespace std;

int t;
int n, m;
int a[maxn][maxn];
int d[maxn][maxn];
vector<int> v[maxn];
int fa[maxn];

void dfs(int i, int x){
    fa[i] = x;
    for(auto to : v[i]) dfs(to, x);
}

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) a[i][j] = d[i][j] = i
        == j ? 0 : 1e9;
        while(m--){
            int x, y, z;
            scanf("%d%d%d", &x, &y, &z);
            a[x][y] = a[y][x] = d[x][y] = d[y][x] = z;
        }
        for(int k = 1; k <= n; k++) for(int i = 1; i <= n; i++) for(int j = 1; j <=
n; j++) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        for(int i = 1; i <= n; i++){
            for(int j = 1; j <= n; j++) v[j].clear();
            for(int j = 1; j <= n; j++) if(i ^ j){
                for(int k = 1; k <= n; k++) if(k ^ j) if(d[i][k] + a[k][j] == d[i]
[j]){
                    v[k].push_back(j);
                    break;
                }
            }
            for(auto to : v[i]) dfs(to, to);
            int ans = 1e9;
            for(int j = 1; j <= n; j++) if(i ^ j){
                if(fa[j] ^ j) ans = min(ans, a[i][j] + d[i][j]);
            }
            for(int j = 1; j <= n; j++) if(i ^ j) for(int k = j + 1; k <= n; k++)
if(i ^ k){
                if(fa[j] ^ fa[k]) ans = min(1ll * ans, 1ll * a[j][k] + d[i][j] +
d[i][k]);
            }
            printf("%d ", ans);
        }
        puts("");
    }
}

```

E5

1

A 小水獭和高精度乘法

时间限制: 3000ms 内存限制: 65536kb

通过率: 171/194 (88.14%) 正确率: 171/1180 (14.49%)

题目描述



小水獭想请你编写一个高精度乘法计算器，给定正整数 A 和 B ，请你输出 $C = AB$ 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 A, B ($1 \leq A, B < 10^{10^5}$)，含义如题目所示。

输出格式

对于每组数据，输出一行一个正整数 $C = AB$ 。

输入样例

```
3
1 1
4 5
1 4
```

输出样例

```
1
20
4
```

```
#include <bits/stdc++.h>
#define maxn 300086

using namespace std;

const double pi = acos(-1);

struct Complex{
    double x, y;
    Complex(double xx = 0, double yy = 0){
        x = xx, y = yy;
    }
};

Complex operator + (Complex a, Complex b){
    return Complex(a.x + b.x, a.y + b.y);
}

Complex operator - (Complex a, Complex b){
    return Complex(a.x - b.x, a.y - b.y);
}

Complex operator * (Complex a, Complex b){
```



```

        return Complex(a.x * b.x - a.y * b.y, a.x * b.y + a.y * b.x);
    }

    int pos[maxn];

    void FFT(Complex *a, int lim, int type){
        for(int i = 0; i < lim; i++) if(i < pos[i]) swap(a[i], a[pos[i]]);
        for(int mid = 1; mid < lim; mid <= 1){
            Complex wn(cos(pi / mid), type * sin(pi / mid));
            for(int i = mid < 1, j = 0; j < lim; j += i){
                Complex w(1, 0);
                for(int k = 0; k < mid; k++, w = w * wn){
                    Complex x = a[j + k], y = w * a[j + k + mid];
                    a[j + k] = x + y;
                    a[j + k + mid] = x - y;
                }
            }
        }
        if(type == -1) for(int i = 0; i < lim; i++) a[i].x /= lim;
    }

    int t;
    Complex a[maxn], b[maxn];
    char s[maxn];
    int n, m;
    int ans[maxn];

    int main(){
        scanf("%d", &t);
        while(t--){
            scanf("%s", s);
            n = strlen(s);
            for(int i = 0; i < n; i++) a[n - i - 1].x = s[i] - '0';
            scanf("%s", s);
            m = strlen(s);
            for(int i = 0; i < m; i++) b[m - i - 1].x = s[i] - '0';
            int lim = 1;
            while(lim <= n + m - 2) lim <= 1;
            for(int i = 0; i < lim; i++) pos[i] = (pos[i >> 1] >> 1) | ((i & 1) ? (lim
>> 1) : 0);
            FFT(a, lim, 1), FFT(b, lim, 1);
            for(int i = 0; i < lim; i++) a[i] = a[i] * b[i];
            FFT(a, lim, -1);
            for(int i = 0; i < maxn; i++) ans[i] = 0;
            for(int i = 0; i < lim; i++){
                ans[i] = (int)(a[i].x + 0.5);
                a[i] = b[i] = {0, 0};
            }
            for(int i = 0; i + 1 < maxn; i++){
                ans[i + 1] += ans[i] / 10;
                ans[i] %= 10;
            }
            bool tag = false;
            for(int i = maxn - 1; ~i; i--){
                if(ans[i]) tag = true;
                if(!ans[i] && !tag) continue;

```

```

        printf("%d", ans[i]);
    }
    puts("");
}
}

```

2

B 旋转卡壳怎么读

时间限制: 1000ms 内存限制: 65536kb

通过率: 147/170 (86.47%) 正确率: 147/1051 (13.99%)

题目描述

教给司发现二维平面上有 n 个点，坐标分别为 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ 。定义两点 (x_i, y_i) 和 (x_j, y_j) 的欧几里得距离为 $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ 。教给司要求同学们求出任意两点间最大欧几里得距离，即：

$$\max_{1 \leq i < j \leq n} \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

输入格式

第一行一个正整数 n ($2 \leq n \leq 10^5$)，表示点的数量。

接下来 n 行，每行两个至多六位小数的浮点数 x_i, y_i ($0 \leq |x_i|, |y_i| \leq 5 \times 10^5$)，表示第 i 个点的坐标。

输出格式

输出一行一个浮点数，保留六位小数，表示任意两点间最大欧几里得距离。

输入样例

```

4
0 1
1 0
1 1
-1 -1

```

输出样例

```

2.828427

```

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
#include<stdbool.h>
#include<vector>
#include<algorithm>
#include<iostream>
using namespace std;
typedef long long ll;
const int N=200005;
const double eps=1e-7;
int getint(){
    char c;int flag=0,num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag?-num:num;
}
double getdb(){double x;scanf("%lf",&x);return x;}
inline int sgn(double x){return (x>eps)-(x<-eps);}
inline double sqr(double x){return x*x;}
struct Point{
    double x,y;
    Point operator + (const Point&b)const{
        return (Point){x+b.x,y+b.y};
    }
}

```

```

    }
    Point operator - (const Point&b)const{
        return (Point){x-b.x,y-b.y};
    }
    double operator * (const Point&b)const{
        return x*b.x+y*b.y;
    }
    double operator ^ (const Point&b)const{
        return x*b.y-y*b.x;
    }
    double dis(Point b){
        return sqrt(sqr(x-b.x)+sqr(y-b.y));
    }
};

vector<Point> p,ans;
bool cmpx(Point a,Point b){
    return sgn(a.x-b.x)==0 ? a.y<b.y : a.x<b.x;
}

vector<Point> ConvexHull(vector<Point> A,int flag){
    int n=A.size(),now=-1;
    vector<Point> ans(n<<1);
    sort(A.begin(),A.end(),cmpx);
    for(int i=0;i<n;i++){
        while(now>0 && sgn((ans[now]-ans[now-1])^(A[i]-ans[now-1]))<flag) now--;
        ans[++now]=A[i];
    }
    int pre=now;
    for(int i=n-2;i>=0;i--){
        while(now>pre && sgn((ans[now]-ans[now-1])^(A[i]-ans[now-1]))<flag) now--;
        ans[++now]=A[i];
    }
    ans.resize(now);return ans;
}

double convexDiameter(vector<Point> A){
    int n=A.size(),now=0;
    double ans=0;
    for(int i=0;i<n;i++){
        now=max(now,i);
        while(1){
            double k1=A[i].dis(A[now%n]),k2=A[i].dis(A[(now+1)%n]);
            ans=max(ans,max(k1,k2));
            if(k2>k1) now++;else break ;
        }
    }
    return ans;
}

int main(){
    int n=getint();
    for(int i=0;i<n;i++){
        double x=getdb(),y=getdb();
        p.push_back((Point){x,y});
    }
    ans=ConvexHull(p,1);
    printf("%.6f\n",convexDiameter(ans));
    return 0;
}

```

}

3



去做题

C 约会大作战-青春版

时间限制: 1000ms 内存限制: 65536kb

通过率: 88/98 (89.80%) 正确率: 88/251 (35.06%)

题目描述

学校正在举办联谊活动! 目前报名的有 n 个男生和 n 个女生。其中第 i 个男生魅力值为 a_i ，只喜欢魅力值不小于 p_i 的女生; 第 i 个女生魅力值为 b_i ，只喜欢魅力值不小于 q_i 的男生。

男生与女生能约会当且仅当两人相互喜欢。作为一个没有钞能力的主办方，the_ignorant 只能希望让尽可能多的人能约会成功，那么最多有多少对情侣能约会成功呢?

输入格式

第一行一个正整数 n ($1 \leq n \leq 400$)，表示男生和女生的数量。

第二行 n 个非负整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$)，表示男生的魅力值。

第三行 n 个非负整数 $p_1, p_2, \dots, p_n$ ($0 \leq p_i \leq 10^9$)，表示男生喜欢的女生魅力值最小值。

第四行 n 个非负整数 $b_1, b_2, \dots, b_n$ ($0 \leq b_i \leq 10^9$)，表示女生的魅力值。

第五行 n 个非负整数 $q_1, q_2, \dots, q_n$ ($0 \leq q_i \leq 10^9$)，表示女生喜欢的男生魅力值最小值。

输出格式

一行一个非负整数，表示最多能有几对情侣约会成功。

输入样例

```
5
3 10 2 1 5
9 7 2 10 1
1 6 10 8 0
9 4 4 1 8
```

输出样例

```
3
```

```
#include<bits/stdc++.h>
using namespace std;
const int N = 500;
char s[N];
int a[N],p[N],b[N],q[N];
bool mat[N][N];
bool vis[N];
int girl[N];
int n;
bool find(int x)
{
    for(int i=1;i<=n;i++)
    {
        if(mat[x][i]&&!vis[i])
        {
            vis[i]=true;
            if(!girl[i]||find(girl[i]))
            {
                girl[i]=x;
                return true;
            }
        }
    }
    return false;
}
int main()
```

```

{

    int t=1;
    while(t-->0)
    {
        scanf("%d",&n);
        for(int i=1;i<=n;i++)
        {
            scanf("%d",&a[i]);
        }
        for(int i=1;i<=n;i++)
        {
            scanf("%d",&p[i]);
        }
        for(int i=1;i<=n;i++)
        {
            scanf("%d",&b[i]);
        }
        for(int i=1;i<=n;i++)
        {
            scanf("%d",&q[i]);
        }
        for(int i=1;i<=n;i++)
        {
            for(int j=1;j<=n;j++)
            {
                if(b[j]>=p[i]&& a[i]>=q[j])
                {
                    mat[i][j]=true;
                }
            }
        }
        int ans=0;
        for(int i=1;i<=n;i++)
        {
            memset(vis,0,sizeof(vis));
            if(find(i))
            {
                ans++;
            }
        }
        printf("%d",ans);
    }
}

```

D 直直的多边形面积

时间限制: 1000ms 内存限制: 65536kb

通过率: 130/141 (92.28%) 正确率: 130/495 (26.26%)

题目描述

Zhoues 最近在研究直直的多边形，直直的多边形是所有边均平行于两条坐标轴之一的简单多边形。
现给出一个直直的多边形，Zhoues 想知道这个直直的多边形的面积。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。
对于每组数据，第一行一个正整数 n ($4 \leq n \leq 4 \times 10^4$)，表示直直的多边形的顶点数。
接下来 n 行，每行两个整数 x_i, y_i ($-5 \times 10^4 \leq x_i, y_i \leq 5 \times 10^4$)，表示直直的多边形一个顶点的坐标。保证顶点坐标按顺时针方向给出，且所有边均平行于两条坐标轴之一。

输出格式

对于每组数据，输出一行一个非负整数，表示直直的多边形的面积。

输入样例

```
1
4
-200 200
200 200
200 -200
-200 -200
```

输出样例

```
160000
```

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define inf 1e100
#define eps 1e-8
//用于浮点数正负判断，根据题目精度修改
const double pi = acos(-1.0); //圆周率
long long sgn(long long x) {
    if(fabs(x)<eps)return 0;
    if(x<0)return -1;
    return 1;
} //判断浮点数正负
long long sqr(long long x) {
    return x*x;    //距离等运算涉及大量平方，简便
}
//使用Point时注意部分函数是返回新Point而非修改本身值
struct Point {
    long long x,y;
    /*构造函数*/
    Point() {}
    Point(long long xx,long long yy) {
        x=xx;
        y=yy;
    }
    /*重载一些点的基础运算符*/
    bool operator == (Point b)const {
        return sgn(x-b.x) == 0 && sgn(y-b.y) == 0;
    }
    bool operator < (Point b)const {
        return sgn(x-b.x)== 0?sgn(y-b.y)<0:x<b.x;
    }
    Point operator -(const Point &b)const {
        return Point(x-b.x,y-b.y);
    }
}
```

```

Point operator +(const Point &b) const {
    return Point(x+b.x,y+b.y);
}
Point operator *(const long long &k) const {
    return Point(x*k,y*k);
}
Point operator /(const long long &k) const {
    return Point(x/k,y/k);
}
//叉积
long long operator ^(const Point &b) const {
    return x*b.y - y*b.x;
}
//点积
long long operator *(const Point &b) const {
    return x*b.x + y*b.y;
}
/*当前点为p, 求角apb大小*/
long long rad(Point a,Point b) {
    Point p = *this;
    return fabs(atan2( fabs((a-p)^(b-p)), (a-p)*(b-p) ));
}
/*逆时针旋转90度*/
Point rotleft() {
    return Point(-y,x);
}
/*顺时针旋转90度*/
Point rotright() {
    return Point(y,-x);
}
//两点距离
long long dis(Point p) {
    return sqrt(sqr(x-p.x)+sqr(y-p.y));
}
long long dis2(Point p) {
    return sqr(x-p.x)+sqr(y-p.y);
}
//原点距离
long long abs() {
    return sqrt(abs2());
}
long long abs2() {
    return sqr(x)+sqr(y);
}
//改变向量长度
Point trunc(long long r) {
    long long l = abs();
    if(!sgn(l))return *this;
    r /= l;
    return Point(x*r,y*r);
}
//单位化
Point unit() {
    return *this/abs();
}
//IO

```

```

void input() {
    scanf("%lld%lld",&x,&y);
}
void output() {
    printf("%lld %lld\n",x,y);
}
//绕着p点逆时针旋转angle
Point rotate(Point p,long long angle) {
    Point v = (*this) - p;
    long long c = cos(angle), s = sin(angle);
    return Point(p.x + v.x*c - v.y*s,p.y + v.x*s + v.y*c);
}
};

//多边形面积，需保证A逆时针
long long area(vector<Point> A) {
    long long ans = 0;
    for (int i = 0; i < A.size(); i++) ans += (A[i]^A[(i + 1) % A.size()]);
    return ans / 2;
}

long long n,cnt;
vector<Point> p;
int main() {
    //freopen("2.in","r",stdin);
    int t;
    scanf("%d",&t);
    while(t--) {
        scanf("%lld",&n);
        p.clear();
        p.resize(n);
        for(int i = 0; i<n; i++) {
            p[i].input();
        }
        printf("%lld\n",-area(p));
    }
}

```


题目描述

观赛完运动会之后，Zhoues 需要立马回去钢厂上班，由于北航到钢厂的路径有很多条，他希望选择一条最短的！
但是貌似最短的路径有很多条，Zhoues 想知道有多少条最短的路径呢？
形式化来说，给一个 n 个点 m 条边的无向图，每条边的权值均为 1，求从点 s 到点 t 的最短路数量，两条最短路不同当且仅当包含了不同的边。由于答案可能很大，你只需要输出答案对 998 244 353 取模后的结果。

输入格式

第一行四个正整数 n, m, s, t ($1 \leq n \leq 10^6, 1 \leq m \leq 1.6 \times 10^6, 1 \leq s, t \leq n$)，含义同题目描述。
接下来 m 行，每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示第 i 条边 (u_i, v_i) ，所有边的权值均为 1。
图中有可能存在重边和自环，保证至少存在一条从点 s 到点 t 的路径。

输出格式

输出一行一个正整数，表示从点 s 到点 t 的最短路数量对 998 244 353 取模后的结果。

输入样例

```
5 7 1 5
1 2
1 3
2 4
3 4
2 3
4 5
4 5
```

输出样例

```
4
```

```
#include<cstdio>
#include<cstring>
#include<queue>
#define maxn 10000006
using namespace std;
int n,m,s,t;
int dis[maxn];
int ans[maxn];
struct Edge{
    int t,nxt,w;
}edge[maxn*4];
int book[maxn];
int head[maxn],tot=0;
priority_queue< pair<int,int> > hep;
void add(int st,int to,int we){
    edge[tot].t=to;
    edge[tot].nxt=head[st];
    edge[tot].w=we;
    head[st]=tot;
    tot++;
}
void init(){
    scanf("%d %d %d %d",&n,&m,&s,&t);
    memset(book,0,sizeof(book));
    memset(head,-1,sizeof(head));
    memset(dis,0x3f,sizeof(dis));
    memset(ans,0,sizeof(ans));
    for(int i=1;i<=m;i++){
        int a,b;
        scanf("%d %d",&a,&b);
        add(a,b,1);
        add(b,a,1);
    }
}
```

```

    }
    dis[s]=0;
    ans[s]=1;
    return;
}

void dij(){
    hep.push(make_pair(0-dis[s],s));
    while(!hep.empty()){
        int np=hep.top().second;hep.pop();
        if(book[np]) continue;
        book[np]=1;
        for(int i=head[np];i!=-1;i=edge[i].nxt){
            int w=edge[i].w;
            int t=edge[i].t;
            if(dis[t]>dis[np]+w){
                dis[t]=dis[np]+w;
                ans[t]=ans[np];
                hep.push(make_pair(0-dis[t],t));
            }
            else if(dis[t]==dis[np]+w){
                ans[t]+=ans[np];
                ans[t]%=998244353;
            }
        }
    }
    printf("%d\n",ans[t]);
    return;
}

int main(){
    init();
    dij();
    return 0;
}

```

F 剪刀石头布！

时间限制：1000ms 内存限制：65536kb

通过率：10/18 (55.56%) 正确率：10/51 (19.61%)

题目描述

BellalaBella 发现了一个剪刀石头布的新玩法，初始给定一个长度为 n 的剪刀石头布序列 S ，其中只有 S, R, P 三种字符，分别表示剪刀、石头与布。游戏会进行 $n - 1$ 轮，每一轮会根据上一轮的序列生成一个新的剪刀石头布序列，每一轮操作按如下方式进行：

- 设上一轮的序列（若为第一轮则为初始给定的序列）为 S ，这一轮生成的新序列为 T ，有 $|T| = |S| - 1$ 。
- 对于 $i = 1, 2, \dots, n - 1$ ，如果 $S_i = S_{i+1}$ 则有 $T_i = S_i$ ，否则 T_i 为 S_i 和 S_{i+1} 进行普通剪刀石头布游戏获胜的一方所对应的字符。

Bellalabella 发现最后一轮所生成的序列仅包含一个字符，她想知道具体是哪个字符。

输入格式

第一行一个正整数 t ($1 \leq t \leq 50$)，表示数据组数。

对于每组数据，一行一个由 S, R, P 组成的字符串 S ($1 \leq |S| \leq 10^6$)，含义同题目描述。

对于所有数据，保证 $|S|$ 之和不超过 3×10^6 。

输出格式

对于每组数据，输出一行一个字符，表示最后一轮生成序列所包含的字符。

输入样例

1
SRP

输出样例

P

```
#include <bits/stdc++.h>
#define maxn 1000086

using namespace std;

typedef long long ll;

int n, m;
vector<pair<int, int>> now;

char s[maxn];

int gettp(char x){
    if(x == 'S') return 0;
    if(x == 'R') return 1;
    return 2;
}

char getans(int x){
    if(x == 0) return 'S';
    if(x == 1) return 'R';
    return 'P';
}

bool check(int a, int b){
    if(a == 0 && b == 2) return 1;
    if(a == 1 && b == 0) return 1;
    if(a == 2 && b == 1) return 1;
    return 0;
}

void insert(pair<int, int> x){
    if(now.empty()) now.push_back(x);
```

```

else{
    pair<int, int> y = now[now.size() - 1];
    now.pop_back();
    if(y.second == x.second){
        y.first += x.first;
        now.push_back(y);
    }else{
        if(check(x.second, y.second)){
            x.first += y.first;
            insert(x);
        }
        else{
            now.push_back(y);
            now.push_back(x);
        }
    }
}
}

int main(){
    int t;
    scanf("%d", &t);
    while(t--){
        scanf("%s", s + 1);
        n = strlen(s + 1);
        now.clear();
        for(int i = 1; i <= n; i++){
            int tp = gett(s[i]);
            insert({1, tp});
        }
        printf("%c\n", getans(now[0].second));
    }
}

```

7

G 重建猫娘乐园

时间限制: 1000ms 内存限制: 131072kb

通过率: 7/7 (100.00%) 正确率: 7/12 (58.33%)

题目描述

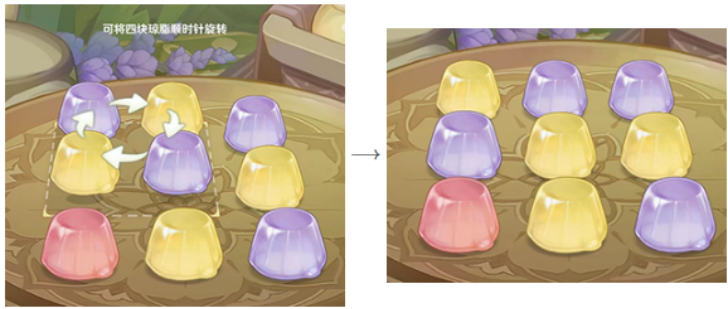
XIAO7 准备利用新抓的草兽来重建猫娘乐园，为了提高重建效率，她需要将「花花琼脂」调配成指定的组合以供草兽吸收，来激发草兽的潜能。请你帮她！

「花花琼脂」有三种颜色，分别用 0, 1, 2 表示，排成 3×3 的矩阵，你可以通过以下三种操作，调整「花花琼脂」的排列顺序，直至其变成指定的组合。

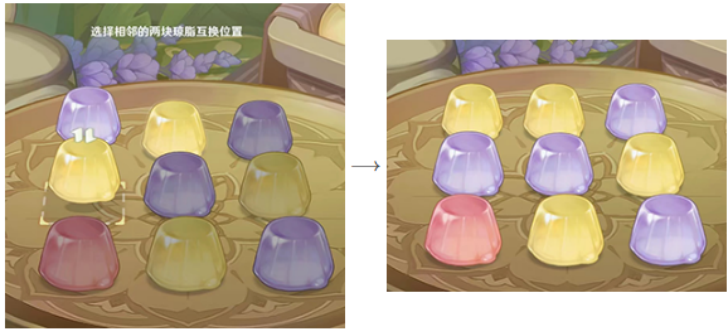
1. 「旋转」：花费 a 枚摩拉，选择任意一个 2×2 的子矩阵，将其顺时针旋转 90° 。例如：

「花花琼脂」有三种颜色，分别用 $0, 1, 2$ 表示，排成 3×3 的矩阵，你可以通过以下三种操作，调整「花花琼脂」的排列顺序，直至其变成指定的组合。

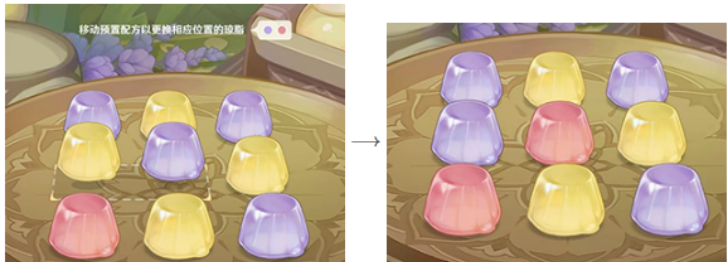
1. 「旋转」：花费 a 枚摩拉，选择任意一个 2×2 的子矩阵，将其顺时针旋转 90° 。例如：



2. 「对换」：花费 b 枚摩拉，选择相邻的两个「花花琼脂」，并交换它们的位置。例如：



3. 「预制」：花费 c 枚摩拉，选择一个 $n \times m$ 的子矩阵，将它预制成指定的样式。例如，若预制指定样式为：



由于重建猫娘乐园的花岗非常大 XIAO7 需要尽可能的节俭，所以请你告诉她，她需要花费的最少摩拉数。

输入格式

第一行三个正整数 a, b, c ($1 \leq a, b, c \leq 10^6$)，分别表示「旋转」、「对换」、「预制」三种操作分别需要花费的摩拉数。

第二行两个正整数 n, m ($1 \leq n, m \leq 3$)，表示「预制」操作可选择的子矩阵大小。

接下来一个 $n \times m$ 的矩阵，表示「预制」操作的指定样式。

接下来一个 3×3 的矩阵，表示「花花琼脂」的初始排列。

接下来一个 3×3 的矩阵，表示「花花琼脂」的最终目标排列。

上述三个矩阵的元素均在 $\{0, 1, 2\}$ 中。

输出格式

一行一个非负整数，表示 XIAO7 最少需要花费的摩拉数。

输入样例

```
1 1 1
3 1
0
0
0
0 0 0
1 1 1
2 2 2
1 0 0
1 0 2
2 2 1
```

输出样例

```
2
```

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
```

```

#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<ctime>
#include<map>
#include<assert.h>
#include<queue>
using namespace std;
typedef long long ll;
typedef pair<ll, int> pi;
const int N = 20005;
const int M = 1000005;
const ll inf = 1ll << 60;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
const int d[4][2] = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};
int cnt, fir[N], tar[M], nxt[M], w[M];
ll dis[N]; bool vis[N];
int Hash(int a[][3]) {
    int h = 0;
    for(int i = 0; i < 3; i ++){
        for(int j = 0; j < 3; j ++){
            h = h * 3 + a[i][j];
        }
    }
    return h;
}
void reHash(int h, int a[][3]) {
    for(int i = 2; i >= 0; i --){
        for(int j = 2; j >= 0; j --){
            a[i][j] = h % 3, h /= 3;
        }
    }
}
void rotate(int x[][3], int y[][3], int si, int sj) {
    for(int i = 0; i < 3; i ++){
        for(int j = 0; j < 3; j ++){
            y[i][j] = x[i][j];
        }
    }
    // si, sj      si, sj + 1
    // si + 1, sj  si + 1, sj + 1

    y[si][sj + 1] = x[si][sj];
    y[si + 1][sj + 1] = x[si][sj + 1];
    y[si + 1][sj] = x[si + 1][sj + 1];
    y[si][sj] = x[si + 1][sj];
}
void link(int a, int b, int c) {
    tar[++ cnt] = b, w[cnt] = c;
    nxt[cnt] = fir[a], fir[a] = cnt;
}
bool inarea(int x, int y) {
    return x >= 0 && x < 3 && y >= 0 && y < 3;
}

priority_queue<pi, vector<pi>, greater<pi> > q;

```

```

void dij(int x) {
    for(int i = 0; i < 19683; i++)
        dis[i] = inf;
    q.push(make_pair(dis[x] = 0, x));
    while(! q.empty()) {
        x = q.top().second, q.pop();
        if(vis[x]) continue ;
        vis[x] = 1;

        for(int i = fir[x]; i; i = nxt[i])
            if(dis[tar[i]] > dis[x] + w[i]) {
                dis[tar[i]] = dis[x] + w[i];
                q.push(make_pair(dis[tar[i]], tar[i]));
            }
    }
}

int main() {
    int x[3][3], y[3][3], color[3][3];
    int a = getint(), b = getint(), c = getint();

    int n = getint(), m = getint();
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            color[i][j] = getint();

    for(int i = 0; i < 3; i++)
        for(int j = 0; j < 3; j++)
            x[i][j] = getint();

    int st = Hash(x), Ed = 19683;
    for(int s = 0; s < Ed; s++) {
        reHash(s, x);

        // Ðýxª
        rotate(x, y, 0, 0), link(s, Hash(y), a);
        rotate(x, y, 0, 1), link(s, Hash(y), a);
        rotate(x, y, 1, 0), link(s, Hash(y), a);
        rotate(x, y, 1, 1), link(s, Hash(y), a);

        // ¶Œ»»
        for(int i = 0; i < 3; i++)
            for(int j = 0; j < 3; j++)
                for(int k = 0; k < 3; k++)
                    if(inarea(i + d[k][0], j + d[k][1])) {
                        swap(x[i][j], x[i + d[k][0]][j + d[k][1]]);
                        link(s, Hash(x), b);
                        swap(x[i][j], x[i + d[k][0]][j + d[k][1]]);
                    }

        // ŒªŒ
        for(int i = 0; i + n - 1 < 3; i++)
            for(int j = 0; j + m - 1 < 3; j++) {
                for(int di = 0; di < n; di++)
                    for(int dj = 0; dj < m; dj++) {
                        y[di][dj] = x[i + di][j + dj];
                        x[i + di][j + dj] = color[di][dj];
                    }
            }
    }
}

```

```

    }
    link(s, Hash(x), c);

    for(int di = 0; di < n; di ++)
        for(int dj = 0; dj < m; dj ++)
            x[i + di][j + dj] = y[di][dj];
    }
}

dij(st);

for(int i = 0; i < 3; i ++)
    for(int j = 0; j < 3; j ++)
        x[i][j] = getint();
//cout << dis[Hash(x)] << '\n';
printf("%lld\n", dis[Hash(x)]);

return 0;
}

```

8

H 小水獭和科学抗疫 (Δ 版)

时间限制: 3000ms 内存限制: 65536kb

通过率: 21/22 (95.45%) 正确率: 21/43 (48.84%)

题目描述



小水獭幼儿园出现了疫情!

小水獭幼儿园可以被抽象为一个数轴, 其中共有 n 名学生, 第 i 名学生位于坐标 x_i , 任意两名学生的坐标均不相同。对于每名学生均有 $\frac{1}{2}$ 的概率被感染, 且每名学生是否被感染相互独立。

基于最新的科学研究, 如果第 i 名学生和第 j 名学生都被感染 ($i < j$), 那么需要额外投入 $2^{x_i - x_j}$ 的核酸费用。设核酸费用之和的数学期望为 C , 容易证明 $4C$ 是一个整数, 请你求解 $4C$ 对 998 244 353 取模后的结果。

同学们众志成城, 根据科学精准防控, 一定能战胜疫情!

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$), 表示数据组数。

对于每组数据, 第一行一个正整数 n ($2 \leq n \leq 10^5$), 含义如题目所示。

第二行 n 个正整数 $x_1, x_2, \dots, x_n$ ($1 \leq x_i \leq 10^5$), 含义如题目所示。保证 $x_i \neq x_j$ 对 $1 \leq i < j \leq n$ 成立。

输出格式

对于每组数据, 输出一行一个非负整数表示 $4C$ 对 998 244 353 取模后的结果。

输出格式

对于每组数据, 输出一行一个非负整数表示 $4C$ 对 998 244 353 取模后的结果。

输入样例

```

2
4
1 2 4 8
3
1 50000 100000

```

输出样例

```

222
227493521

```

```
#include<bits/stdc++.h>
```



```

using namespace std;
typedef long long LL;
const int maxn = 1e5 + 5;
const LL mod = 998244353;

LL x[maxn], POW[maxn], ans[maxn];

int main() {
    POW[0] = 1;
    for(int i=1; i<maxn; i++) POW[i] = (POW[i-1] * 2) % mod;

    int T, n;
    scanf("%d", &T);
    while(T--) {
        scanf("%d", &n);
        for(int i=1; i<=n; i++) scanf("%lld", &x[i]);
        sort(x+1, x+1+n);
        ans[n-1] = POW[x[n]-x[n-1]];
        for(int i=n-2; i>=1; i--) ans[i] = (POW[x[i+1]-x[i]] * (1 + ans[i+1])) %
mod;

        int res = 0;
        for(int i=1; i<=n-1; i++) res = (res + ans[i]) % mod;
        printf("%lld\n", res);
    }

    return 0;
}

```

9

I 小水獭和紧急演习

时间限制: 1000ms 内存限制: 65536kb

通过率: 125/128 (97.66%) 正确率: 125/330 (37.88%)

题目描述



加帕里幼儿园正在预防出现宇宙爆炸!

加帕里幼儿园可以被抽象为一个 m 维欧氏空间, 其中有 n 个实验室, 第 i 个实验室的坐标为 $(x_{i,1}, x_{i,2}, \dots, x_{i,m})$.

加帕里幼儿园即将紧急演习, 为计算财政支出, 他们请小水獭帮忙计算一下加帕里幼儿园的直径, 即任意两所实验室之间曼哈顿距离的最大值。

在 m 维欧氏空间中, 两点 $(x_1, x_2, \dots, x_m)$ 和 $(y_1, y_2, \dots, y_m)$ 的曼哈顿距离为:

$$\sum_{i=1}^m |x_i - y_i|$$

小水獭的浏览器弹窗了, 无法前往实地计算, 你能帮帮它嘛?

输入格式

第一行一个正整数 t ($1 \leq t \leq 20$), 表示数据组数。

对于每组数据, 第一行两个正整数 n, m ($2 \leq n \leq 10^4, 1 \leq m \leq 5$), 含义同题目描述。

接下来 n 行, 第 i ($1 \leq i \leq n$) 行 m 个整数 $x_{i,1}, x_{i,2}, \dots, x_{i,m}$ ($-10^9 \leq x_{i,j} \leq 10^9$), 表示第 i 个实验室的坐标。保证没有两个不同的实验室位于相同的位置。

输出格式

对于每组数据, 输出一行一个正整数, 表示任意两所实验室之间曼哈顿距离的最大值。

输出格式

对于每组数据，输出一行一个正整数，表示任意两所实验室之间曼哈顿距离的最大值。

输入样例

```
2
2 1
1
2
3 2
1 1
2 2
3 3
```

输出样例

```
1
4
```

```
#include <stdio.h>
#define max(a, b) ((a) > (b) ? (a) : (b))

int main()
{
    int t, n, m, a[5], i, j, k;
    long long val[32], x, ans;

    scanf("%d", &t);
    while (t--)
    {
        scanf("%d%d", &n, &m);
        for (i = 0; i < (1 << m); i++)
            val[i] = -1e18;
        for (i = 1; i <= n; i++)
        {
            for (j = 0; j < m; j++)
                scanf("%d", &a[j]);
            for (j = 0; j < (1 << m); j++)
            {
                x = 0;
                for (k = 0; k < m; k++)
                {
                    if (j & (1 << k))
                        x += a[k];
                    else
                        x -= a[k];
                }
                val[j] = max(val[j], x);
            }
        }
        ans = -1e18;
        for (i = 0; i < (1 << m); i++)
            ans = max(ans, val[i] + val[i ^ ((1 << m) - 1)]);
        printf("%lld\n", ans);
    }

    return 0;
}
```

J 小水獭和机动冈达战士

时间限制: 1000ms 内存限制: 65536kb

通过率: 7/8 (87.50%) 正确率: 7/24 (29.17%)

题目描述



小水獭正在重温「机动冈达战士：铁血的达因斯尼夫」，它决定代替西洛驾驶「流星号」前往目标点狙击莱斯特·艾里安。

小水獭所在的战场可以被抽象成一个 $n \times m$ 的矩阵，它现在处于 $(1, 1)$ ，目标点为 (n, m) ，除了这两点外，每一点都有一个敌方机动战士，位于 i 行 j 列的敌方机动战士的战斗力为 $a_{i,j}$ 。

小水獭每次可以向下移动或向右移动，并击败沿途的所有敌方机动战士，它希望一路上击败的敌方机动战士的战斗力之和最小，这样才能保证最高的战力来进行狙击。

然而，残忍的冈达使用冈达瞄准了一个子矩阵区域，使得小水獭不能经过这个子矩阵，否则它会被立刻粉碎。

现在，奥尔如提前获知了 q 种冈达可能选择的子矩阵，三日月希望你可以帮助小水獭算一算：对于每种情况，它能否到达目标点；如果可以到达目标点，那么如果选择最优路线，一路上击败的敌方机动战士的战斗力之和的最小值是多少。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行三个正整数 n, m, q ($2 \leq n, m \leq 200, 1 \leq q \leq 5 \times 10^4$)，含义同题目描述。

接下来 n 行，第 i ($1 \leq i \leq n$) 行 m 个非负整数 $a_{i,1}, a_{i,2}, \dots, a_{i,m}$ ($0 \leq a_{i,j} \leq 10^9$)，含义同题目描述。保证 $a_{1,1} = a_{n,m} = 0$ 。

接下来 q 行，第 i ($1 \leq i \leq q$) 行四个正整数 x_1, y_1, x_2, y_2 ($1 \leq x_1 \leq x_2 \leq n, 1 \leq y_1 \leq y_2 \leq m$)，表示第 i 种情况冈达所选择的子矩阵的左上角 (x_1, y_1) 和右下角 (x_2, y_2) 。

输出格式

对于每组数据，输出 q 行，第 i ($1 \leq i \leq q$) 行一个整数，表示对于第 i 种情况，如果小水獭可以到达目标点，输出它一路上击败的敌方机动战士的战斗力之和最小值；否则，输出 -1 。

输出格式

对于每组数据，输出 q 行，第 i ($1 \leq i \leq q$) 行一个整数，表示对于第 i 种情况，如果小水獭可以到达目标点，输出它一路上击败的敌方机动战士的战斗力之和最小值；否则，输出 -1 。

输入样例

```
2
3 3 3
0 1 2
3 4 5
6 7 0
2 1 2 2
1 1 2 2
1 2 3 2
4 4 3
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 0
2 2 3 3
2 1 2 3
3 2 3 4
```

输出样例

```
8
-1
-1
24
24
51
```

```
#include <stdio.h>

#define maxn 205
#define min(a, b) (a < b ? a : b)

typedef long long ll;

int main()
{
    int a[maxn][maxn], t, n, m, q, i, j, x1, y1, x2, y2;
    ll f[maxn][maxn], g[maxn][maxn], pre[maxn][maxn], suf[maxn][maxn], ans;

    scanf("%d", &t);
    while (t--)
    {
        scanf("%d%d%d", &n, &m, &q);
        for (i = 1; i <= n; i++)
```

```

{
    for (j = 1; j <= m; j++)
    {
        scanf("%d", &a[i][j]);
        f[i][j] = g[i][j] = 1e18;
    }
}
for (i = 1; i <= n; i++)
{
    for (j = 1; j <= m; j++)
    {
        if (i == 1 && j == 1) f[i][j] = 0;
        if (i - 1 > 0) f[i][j] = min(f[i][j], f[i - 1][j] + a[i][j]);
        if (j - 1 > 0) f[i][j] = min(f[i][j], f[i][j - 1] + a[i][j]);
    }
}
for (i = n; i; i--)
{
    for (j = m; j; j--)
    {
        if (i == n && j == m) g[i][j] = 0;
        if (i + 1 <= n) g[i][j] = min(g[i][j], g[i + 1][j] + a[i][j]);
        if (j + 1 <= m) g[i][j] = min(g[i][j], g[i][j + 1] + a[i][j]);
    }
}
for (i = 1; i <= n; i++)
{
    for (j = 1; j <= m; j++)
    {
        if (j == 1)
            pre[i][j] = f[i][j] + g[i][j] - a[i][j];
        else
            pre[i][j] = min(pre[i][j - 1], f[i][j] + g[i][j] - a[i][j]);
    }
    for (j = m; j; j--)
    {
        if (j == m)
            suf[i][j] = f[i][j] + g[i][j] - a[i][j];
        else
            suf[i][j] = min(suf[i][j + 1], f[i][j] + g[i][j] - a[i][j]);
    }
}
while (q--)
{
    scanf("%d%d%d%d", &x1, &y1, &x2, &y2);
    if (x2 - x1 + 1 == n || y2 - y1 + 1 == m || (x1 == 1 && y1 == 1) ||
(x2 == n && y2 == m))
    {
        puts("-1");
        continue;
    }
    ans = 1e18;
    if (x2 < n && y1 > 1) ans = min(ans, pre[x2 + 1][y1 - 1]);
    if (x1 > 1 && y2 < m) ans = min(ans, suf[x1 - 1][y2 + 1]);
    printf("%lld\n", ans);
}

```

```
}  
  
    return 0;  
}
```